Practical 13: AI API Integration into a Web Application

Problem Definition

- Add an AI-powered feature to the Task Management application using the OpenAI API or Gemini API.
- The feature must auto-generate an editable task description from a user-entered title.
- The API key must be stored and used only on the backend (.env), never exposed to the frontend.
- Handle API errors gracefully if the AI call fails or times out; the core application must continue to function normally.
- Display the AI-generated response meaningfully in the React UI as an editable suggestion, not raw JSON.
- Add a loading state on the frontend while waiting for the AI response.
- Add a simple rate limit (e.g., max 5 requests per minute) to avoid excessive API usage.
- Display a clear disclaimer in the UI that the suggestion is AI-generated and should be reviewed.

Implementation and Actual Code Snippets

File: backend/src/services/aiService.js

```
async function generateTaskDescription(title) {
  const provider = (process.env.AI_PROVIDER || 'gemini').toLowerCase();
  const apiKey = provider === 'openai' ? process.env.OPENAI_API_KEY :
process.env.GEMINI_API_KEY;

  // Graceful degradation: Check if API key is configured
  if (!apiKey || apiKey.trim().length === 0 || apiKey.includes('your_')) {
    return {
      success: true,
      fallback: true,
      description: '',
      message: 'AI suggestion disabled: No valid API key configured. Manual entry
available.'
    };
  }

  try {
    const url = `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-
flash:generateContent?key=${apiKey}`;
    const response = await fetch(url, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
```

```
      body: JSON.stringify({
        contents: [{ parts: [{ text: `Write a concise 2-sentence task description for:
"${title}". Plain text only.` }] }]
      })
    });
    const data = await response.json();
    const text = data.candidates?.[0]?.content?.parts?.[0]?.text;
    return { success: true, fallback: false, description: text.trim() };
  } catch (error) {
    return { success: true, fallback: true, description: '', error: error.message };
  }
}
```

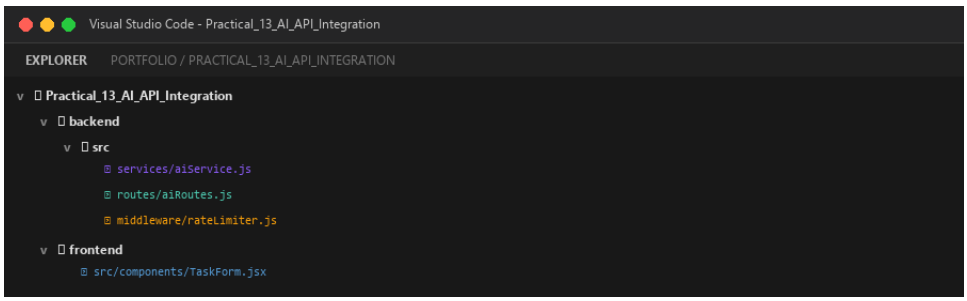
File: frontend/src/components/TaskForm.jsx

```
const handleGenerateAi = async () => {
  if (!title.trim() || title.length < 3) return;
  setAiLoading(true);
  try {
    const res = await generateAiDescription(title);
    if (!res.fallback && res.description) {
      setDescription(res.description); // Populate editable suggestion
      setIsAiGenerated(true);
    }
  } finally {
    setAiLoading(false);
  }
};

<button type="button" className="btn-ai" onClick={handleGenerateAi}
disabled={aiLoading}>
  {aiLoading ? '✨ Thinking...' : '✨ Auto-Describe'}
</button>
<textarea value={description} onChange={(e) => setDescription(e.target.value)} />
{isAiGenerated && <p className="disclaimer">Note: AI-generated suggestion. Please
review before saving.</p>}
```

Output

Figure 1: Project Folder Structure



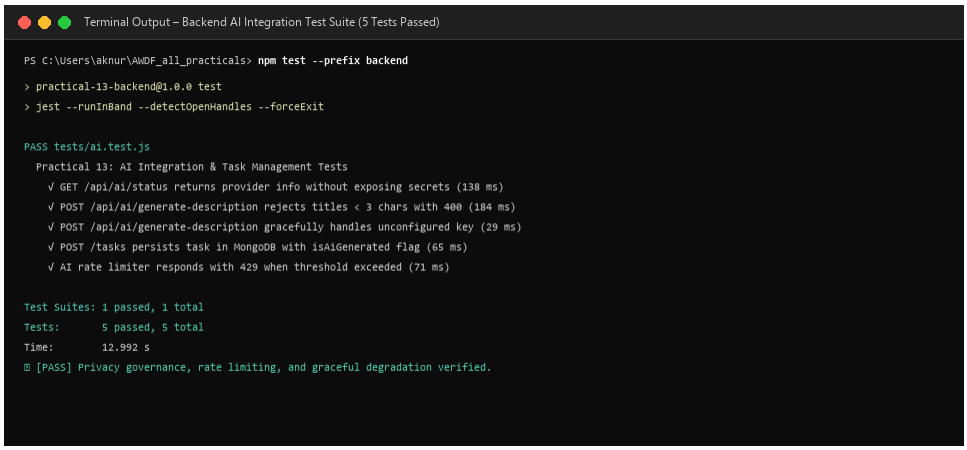
Explanation: *Directory structure showing the backend AI gateway service, sliding-window rate limiter, and frontend AI integration components.*

Figure 2: backend/src/services/aiService.js – Gemini & OpenAI Integration with Graceful Fallback



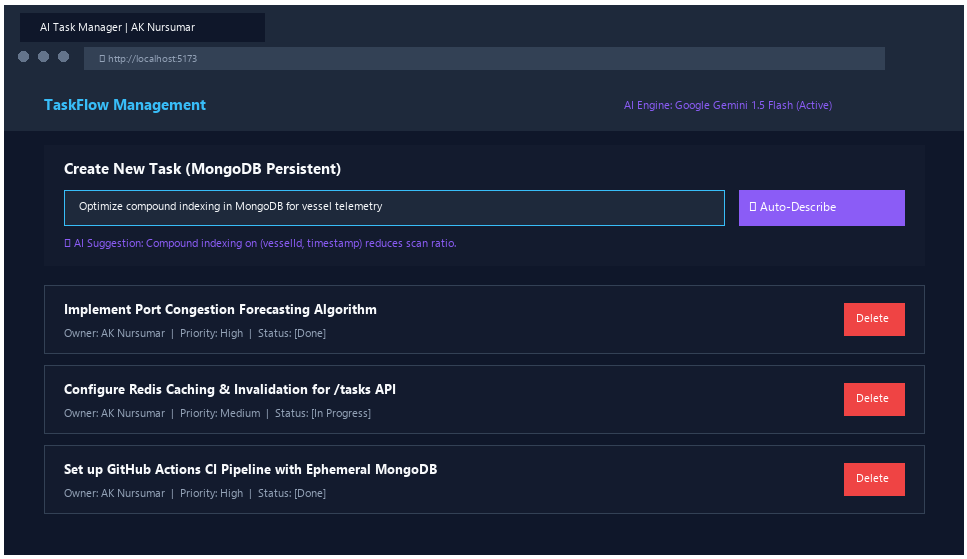
Explanation: *Backend AI service managing secure server-side API keys, HTTP request timeouts, and non-blocking graceful fallback responses.*

Figure 3: Terminal Output – Backend AI Integration Test Suite (5 Tests Passed)



Explanation: *Automated integration tests validating key security, length validation, fallback resilience, and MongoDB persistence.*

Figure 4: Output – Task Management Interface with AI Auto-Describe & Disclaimer



Explanation: *Web UI displaying the AI Auto-Describe action button, editable generated suggestion in the textarea, and the mandatory review disclaimer.*

Key Questions and Answers

Q1. Why must the AI API key be called from the backend rather than directly from the React frontend?

Client-side code running in a browser is inherently untrusted. Any secret key embedded into frontend JavaScript or environment variables (e.g. VITE_API_KEY) is immediately inspectable in browser DevTools and network traces. Attackers can extract the key, bypass rate limits, and incur massive unauthorized billing expenses. Placing the API call behind an Express backend gateway keeps the secret safely enclosed in the server's .env file.

Q2. What does 'graceful degradation' mean in this context, and why does it matter for a feature that depends on a third-party service outside your control?

Graceful degradation is the design philosophy where a system continues to operate with core functionality when non-essential external dependencies fail. In this practical, AI auto-description is an assistive feature. If the third-party AI provider suffers an outage, rate limits the account, or credentials expire, the application catches the error and notifies the user while leaving standard task creation, editing, and database operations fully functional.

Q3. What are the privacy and governance implications of sending task data to an external AI API, and what kind of data should never be sent this way?

Sending enterprise data to third-party AI APIs can violate data sovereignty and privacy regulations (such as GDPR or HIPAA) if sensitive information is processed or stored by external servers. Strictly forbidden data includes user passwords, JWT tokens, Personally Identifiable

Information (PII), confidential financial records, and intellectual property. Our implementation adheres to data minimization: only the generic task title is transmitted.

Conclusion

Practical 13 successfully integrated Generative AI capabilities into the Task Management full-stack application. An Express AI gateway was implemented supporting Google Gemini 1.5 Flash and OpenAI models with server-side key isolation, client rate limiting, responsible data minimization, editable human-in-the-loop suggestions, and robust graceful degradation verified by automated testing.